



Customer 360 & CAR: Real-Time Streaming Layer — Design Brief

A narrow, named set of fraud features moves from the 24-hour batch CAR to a minutes-scale streaming path – the design the 30/60/90 plan scheduled to begin at Day 90, and the one Part 2 (Section 1.5) argued the platform deliberately wasn't ready to build any earlier.

PART 3 OF 3 — REAL-TIME STREAMING LAYER DESIGN BRIEF

Prepared For	Companion Documents	Status	Scope
Engineering, AI/ML, Product	#1 Architecture ERD #2 Part 2	Draft for review	Streaming architecture for two named fraud features; explicit boundary against the rest of the CAR

1 Real-Time Streaming Layer

1.1 Why Now, and What Moves First

Deliverable 1's execution plan scheduled streaming-layer design to begin at Day 90, deliberately after identity resolution and the golden Customer 360 were proven at Day 30 and Day 60 – not because streaming is hard to build, but because a streaming feature is only as trustworthy as the identity it's keyed to, and Part 2 (Section 1.5) already made the case that retrofitting identity resolution underneath an existing streaming layer would mean re-deriving every fraud-relevant feature the moment a merge gets corrected. That sequencing is now satisfied, which is what makes this design brief timely rather than premature.

This brief covers exactly two features, chosen because they are the ones a 24-hour refresh genuinely cannot serve: `txn_velocity_5min` (a trailing five-minute transaction count per `ecid`) and `new_device_flag` (whether the device attempting a transaction has no prior authenticated binding to that `ecid` in `DEVICE_IDENTITY`). Both are fraud signals that lose almost all of their value if they are a day stale – a velocity spike or an unrecognized device is informative in the next few minutes, not the next morning. No other feature is in scope here; this is not a proposal to move the CAR to streaming.

1.2 Architecture: Ingestion, Processing, Serving

Two event sources already exist upstream and are reused rather than duplicated. Core banking emits a transaction-posted event at the same point it already writes to the source system the nightly batch reads from; Amplitude's authenticated session events are the same ones `BEHAVIORAL_EVENT` already ingests, filtered to the authenticated subset per `DEVICE_IDENTITY`'s existing binding rule. Both event streams carry `ecid`



already resolved – assigned synchronously at KYC/login time, the same way it always has been – so the streaming layer never performs identity resolution of its own; it only ever reads an `ecid` that Deliverable 1, Section 2’s waterfall has already produced.

Ingestion lands both event streams in Amazon Kinesis Data Streams. A small set of narrowly-scoped jobs in Amazon Managed Service for Apache Flink (the AWS-managed Flink runtime, formerly branded Kinesis Data Analytics for Apache Flink) computes the two features: a five-minute sliding window count per `ecid` for velocity, and a stateful lookup against known device bindings for the new-device flag. Computed values are written to a dedicated DynamoDB table, keyed by `ecid`, that the fraud model reads directly at transaction-authorization time – the same single-digit-millisecond read path already used for the token vault, not a new class of datastore introduced for this one purpose.

WHY THIS WAY: This DynamoDB table is deliberately not a new column family on `CAR_SNAPSHOT`. Deliverable 2’s ERD stays exactly as specified – the streaming features live in a separate, narrow serving table that the fraud model joins to the `CAR` by `ecid` at inference time, rather than being merged into the nightly snapshot. Folding them into `CAR_SNAPSHOT` would force the whole table to either wait on a five-minute-fresh input before it can publish, or carry two different freshness guarantees under one schema with no way to tell them apart – exactly the ambiguity `feature_freshness_hours` exists to prevent. Keeping them separate means Deliverable 2 does not need to change for this design to ship.

1.3 Technology Choice

Option	How It Works	Effort to Stand Up	Fit for Mal
Kinesis Data Streams + Managed Service for Apache Flink	Fully managed, serverless ingestion and stream processing; Flink’s native windowing computes the sliding-window count directly	Low — no cluster capacity planning on either the ingestion or processing side	Selected
Amazon MSK Serverless + Flink	Same Flink processing, but on the open Kafka API instead of Kinesis; still no capacity planning to manage	Medium — a second stream-storage paradigm alongside everything else already built on Kinesis-adjacent AWS services	Credible fallback if a future integration specifically needs Kafka-native consumers
DynamoDB Streams + Lambda (hand-rolled)	Lambda reacts to each write, maintaining the five-minute window itself in application state	Lowest — no new ingestion service at all	Acceptable Day-90 prototype only; hand-rolling windowed aggregation in Lambda state is the wrong tool once real load arrives



WHY THIS WAY: Kinesis Data Streams paired with Managed Service for Apache Flink wins for the same reason FindMatches won in Deliverable 1: it is fully managed on both ends, so nothing here requires Mal to plan or operate cluster capacity, and Flink’s windowing is a genuine capability match for what a trailing five-minute count actually is – a stateful streaming aggregation – rather than something to approximate with Lambda invocations reacting to individual writes. This is new infrastructure, unlike FindMatches’ reuse of an already-provisioned Glue Catalog, and it is worth being direct about that: it adds a second managed-streaming dependency on top of an already AWS-concentrated stack, which is the same vendor-concentration trade-off Part 2 (Section 4) already named and the platform already accepts. MSK Serverless remains the credible fallback if a future integration – a partner system, a Kafka-native consumer – specifically needs the open protocol; nothing about this design forecloses that option later, since Flink’s processing logic does not meaningfully change based on which stream storage feeds it.

1.4 What Stays Batch, and Why

Everything else – every other field in CAR_SNAPSHOT, every customer who is not mid-transaction, every model that consumes yesterday’s closing state rather than this minute’s – stays exactly where Part 2 (Section 1.5) put it: on the nightly Glue job, published as SCD Type 2, refreshed once a day. That argument does not change here. A fraud model asking “what did we believe about this customer on a given date” still needs the batch CAR’s auditability, and most of the platform’s actual feature set has no minutes-scale consumer today. This design does not reopen that decision; it adds a narrow, explicitly bounded exception for the two features named in Section 1.1, and nothing else.

1.5 Where This Fits: Identity, Consistency, and Boundaries

The streaming layer inherits the platform’s central invariant rather than working around it: every foreign key points toward ecid, and identity resolution stays upstream of everything else (Deliverable 2’s cardinality rationale). Concretely, that means the streaming features carry exactly the same exposure to a bad merge that Part 2 (Section 1.6) already named for the batch path – if a crosswalk correction re-points an ecid, the DynamoDB feature table has stale entries under the old key until the next event re-keys it, which is a narrower instance of an already-accepted risk, not a new one this design introduces. Consent treatment also follows precedent exactly: both features are computed under the same legitimate-interest basis Deliverable 1 (Section 5) already applies to the CAR’s Transaction Behavior group, so neither is gated by personalization consent.

1.6 Open Risks and What This Brief Does Not Cover

Three things are deliberately left open rather than quietly assumed. First, this design does not specify how the new DynamoDB feature table is wired into the PII tier model – it should inherit Tier 1 tokenized-pseudonym access exactly as the rest of the platform does, but Deliverable 1’s access-control mapping does not yet name it explicitly, and it should before this ships. Second, the five-minute freshness target is a design commitment, not a load-tested number; it has not been measured against Mal’s actual transaction volume. Third, expanding streaming coverage beyond these two features is explicitly out of scope for this brief – any future candidate should be held to the same bar Section 1.1 applied here: a feature only qualifies if a day of staleness measurably destroys its value, not merely because faster would be nicer.